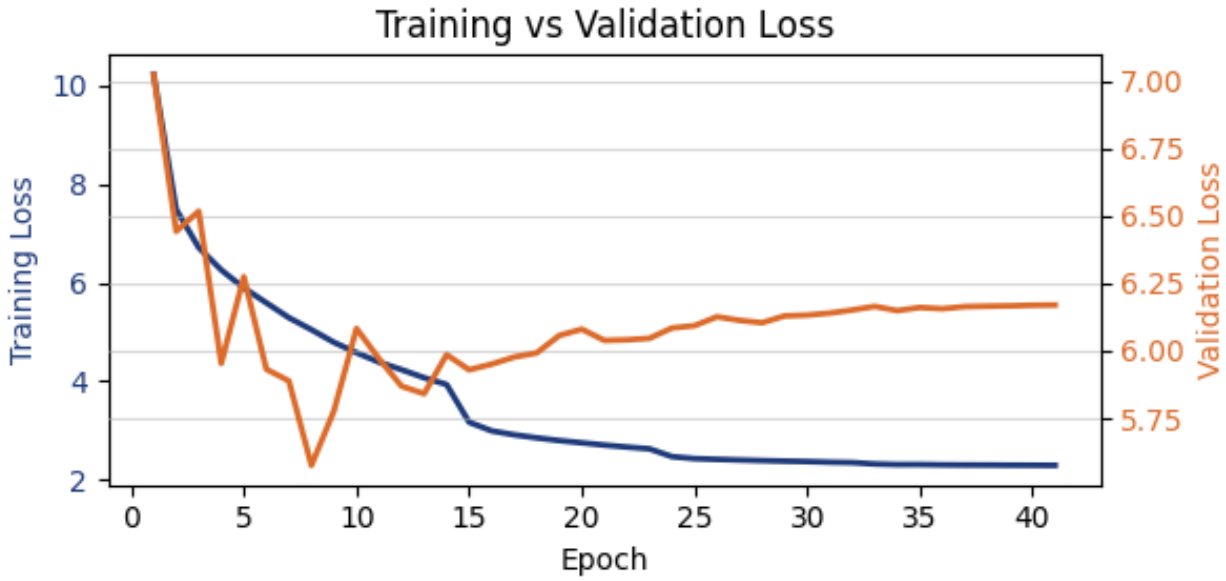
ML to improve streamflow forecasting

Implementations and testing of a new type of model: LSTM

Draft



hydrique.

Hydrique Ingénieurs
Ch. du Rionzi 54 bis, CH-1052 Le Mont-sur-Lausanne
+41 21 784 33 13
www.hydrique.ch info@hydrique.ch

TABLE DES MATIÈRES

1 INTRODUCTION 2

1.1 Context 2
1.1.1 Old way: MLP workflow 2
1.2 Leads to improve our score – Axes of improvement 3
1.3 Data at disposal 3

2 TECHNICAL INFORMATION 4

2.1 Data acquisition: 4
2.1.1 Questions: 4
2.2 Why try LSTM models? 5
2.2.1 LSTM description – Architecture analysis? 5
2.2.2 Typical uses, examples from papers 5
2.2.3 Examples of implementation 6

3 CODE 7

3.1 How each step is done with NH 7
3.2 Tools added 7

4 AR-LSTM 8

4.1 Description 8
4.1.1 Characteristics 8
4.2 Results 9
4.3 Concerns - Questions 9

5 HINDCAST-FORECAST LSTM 11

5.1 Description of the model 11
5.2 Results 12
5.3 Questions 13

6 SEQUENTIAL LSTM USING ONLY RS OUTPUTS 15

6.1 Description 15
6.2 Preliminary results 15

7 NH – OPINION AND WHAT HAVE BEEN DONE 17

7.1 Tools implemented – some of them at least 17
7.2 ML Typical workflow 19

REFERENCES 21

1 Introduction

1.1 Context

Current ML in Hydrique

Hydrique engineers is currently using a MLP model to improve their streamflow forecasts coming out of their software RS. Their MLP has already been optimized multiple times and a score over a +24h horizon has achieved a Mean Absolute Percentage Error (MAPE) of 9.7.

Objectives received

A package with a promising variety of models (NeuralHydrology) is to be tested, to check if better results can be achieved by one of their models. The primary goal is thus to download and become familiar with this package, to test their Auto-Regressive Long-Short Term Memory (AR-LSTM) model.

1.1.1 Old way: MLP workflow

Model

Until now, the Bureau uses a Multi-Layer Perceptron (MLP) model, feeding it sequences of input features and a target (the value at t+24h of the last element of the input sequence) – classical supervised learning.

Main limitation of the MLP model

The main limitation of this method is that the model has now way to keep an history of the previous time steps of the input, it reads it all as a chronologically unordered vector. This issue is solved when going with an LSTM model, that has the property of handling the sequence of input, by taking the time steps one by one, updating its "inner state" dynamically. A description of this type of model will be provided in the next section.

Steps to obtain a prediction

The MLP model was trained to output a single value for each sequence given as input – the prediction at t+N hours, N in {24, 48, 72, 96}. This leads to a major problem, to do an accurate forecast, we had to use a backward rolling window, predicting +1 to +24 hours individually, only the +24h having access to the latest (current) measured values. It works fine (see score table below), but it seems not optimal. A better approach would be to use the measurements until time t to obtain the horizon of +1 to +24 hours.

The MAPE scores obtained with the MLP model are gathered in the Tab. 1.

Horizon	+24h	+48h	+72h	+96h
MAPE [%] Training	9.7	12.1	12.8	13.9
MAPE [%] Validation	9.8	12.16	12.57	13.2

Tab. 1: Scores on the metric MAPE with the four horizons of prediction, using the MLP model.

Remark: These results were obtained for a (relatively short) training and validation period over 2 and 1 years respectively.

Training period: 1.05.2020- 31.10.2022.
validation period: 1.05.2023 – 31.10.2023

1.2 Leads to improve our score – Axes of improvement

Three axes of improvement

To possibly improve our forecast, we can try to lever and work on these axes:

1. Model:
   Which model do we want to use, why and what are the expectations using such model.

2. Once the model is selected, the data can be extracted, organized, and cleaned (pre-processing is essential in Machine Learning). Data preparation depends greatly on the model used.

3. Choice of hyperparameters and training options

1.3 Data at disposal

Inputs

As input, I have access to numerous meteorological stations with measurements, many meteorological forecasting models (I tried ICON-CH1 for the +24h horizon) and the data from our RS simulations. The main issue is the choice of which one to use and the sparsity of some sources (e.g. ICON models that have data only from 2020) – giving a relatively short training period.

I would say that the first important step is thus to define the strategy after having reviewed this report (specially the types of models and which one should be optimized first), and preparing a list of defined inputs. I say it here to be sure that this fact is known by all: changing the input results in having to re-optimize our model. The same is true with changing the architecture or main parameters of our training.

2 Technical information

Motivation of this section

This section's goal is to have a common base of knowledge concerning the technical items of the rest of this report. These points are essential to choose an optimal or at least adapted model and define the process by which the data will go through before being used as input by our model.

2.1 Data acquisition:

The Dataset provided by Josep was used for the first tries but was replaced by a simplified and more specific DS for each main models trained at later time. Indeed, this large dataset was not optimal when organizing and doing small statistical check of our used features. Note that some tests made on this DS was made and it seems like it was in accordance with the data available in 'BD_Backup' on the disk **X**.

For each new main iteration, a new, specific DS is created in my data folder. For example, in my 'massa_2' DS – with which a score of 11 for the MAPE has been obtained – the selected features (measured & forecasted values) were:

- Temperature: Binn, Visp, Eggishorn, Jungfraujoch
- Precipitation: Bruchji, Visp, Fieschertal
- Radiation: Eggishorn, Visp, Jungfraujoch
- Time: hr (=1 at noon and 0 at 0000), hr_sin (sine with hours of a day), daylight (peak during June, sunny hour ratio in a day), doy_sin (sine with day of year)

NOTE:
By default, not all features are used in each model, a combination of them is tested. A lot of combination has been tested, but their performance depends strongly on the model and hyperparameters chosen.

1. Any feedback, advice or list of interesting stations is welcome
2. In addition to these data, a basic Jupyter notebook can be used to add rolling features (sum or average over a given window of previous points).

Another Jupyter notebook can do small verifications and extract main statistical values from each feature (we can for example easily compare the forecasted and measured at a station for a period).

2.1.1 Questions:

- which inputs from RS would we want to implement (or at least test them). Heard from Luca and Johann that it improved the MLP model by a large factor -> sounds interesting and should be explored.
  Once the list is set, the question of the creation and availability (location) of such data

is to be discussed.
Check Model with inputs form RS to see what was tested for now.

- Period available for training:
  For now, my meteorological forecasts are from ICON CH1, meaning the first data available is from the 1st March 2020 at 0000. That shortens greatly our training period. Is there a way to ask for newly created forecasts using the chosen model (that can be changed if a better one is proposed) for a past period?
  I think that increasing our training period (that is of around 3 years now) could be an "easy" way to improve the performance of the model.

Data Preprocessing:

- Was this question already discussed?
  For example, the choice of the scaler (min-max or centre with mean/var), the seasonality from our data, etc.
  It is possible that removing the seasonality of our data and target would allow the model to better react to a variation from a "classical value given this time of a year."

2.2 Why try LSTM models?

Introduction of LSTM models

Long-Short Term Memory models is a type of Recurrent Neural Network (RNN), designed to process sequential data with recurrent connections. They, to cite one of their properties, allow to retain information from their previous Inputs or Outputs.

Pro's of LSTM with respect to RNN for TS forecasting

LSTM architectures are imposing themselves in the domain of Time Series among other RNN mainly by their capability of solving the problem of exploding or vanishing gradients when training with long time-series, of the classical RNN architecture. Indeed, the problem of the gradient hinder the model to correctly learn and causes the training to be inefficient.

2.2.1 LSTM description – Architecture analysis?

Gates:

3 gates, there to decide what to keep from earlier state, …

Go through the sequence time step by time step, chronological importance of the sequence, must be ordered -> sliced correctly.

Outputs:

One-to-One

Seq-to-Seq

2.2.2 Typical uses, examples from papers

LSTM are used in many fields:

- Natural Language Processing: Machine translation, text generation, sentiment analysis, and speech recognition.
- Time Series Analysis: Stock market prediction, weather forecasting, and signal processing.
- Other tasks: Handwriting recognition, image captioning, and video analysis.

2.2.3 Examples of implementation

Sources and examples availability

A large variety of interesting implementations and presentations of LSTM models is available on the internet, some of them are gathered in the reference section [reference to the section]. An interesting example of implementation of LSTM models is presented here: https://machinelearningmastery.com/how-to-develop-lstm-models-for-time-series-forecasting/.

Choice of the model

The diversity of possible models, all with their own characteristics, motivate even more the work that must be done in advance of the development phase to choose wisely a model to test.

3 Code

3.1 How each step is done with NH

Check code hehe

3.2 Tools added 😊

Too many to list

4 AR-LSTM

Context

I was first asked to interest myself with this model in the NH package because it could have some practical properties for our streamflow forecast capability and usage.

Sources:
[https://www.tensorflow.org/tutorials/structured_data/time_series#advanced_autoregressive_model,]

4.1 Description

This model's interesting feature lies in the fact that it should be able to produce output(s) even in the case where the input is not anymore available.

As a concrete case, it should be able to work on a sequence of measure features (context window taking measured values of essential dynamical forcings – precipitation, temperature) producing an output at each time step (that could be compared to actual measure of streamflow in this window). The second phase would then consist on feeding it only its previous outcome on a sequence of a given length (our wanted horizon of prediction).

A classical architecture of an AR-LSTM model is presented in Fig. 1 (source in caption).

Classical AR-LSTM structure

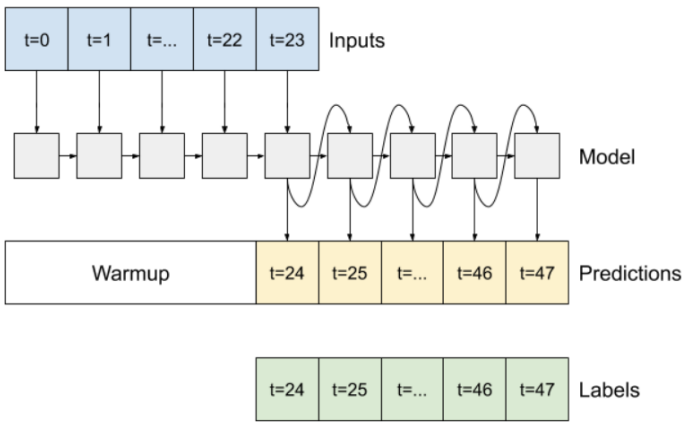


Fig. 1: Diagram of a typical Auto-Regressive LSTM Model
https://www.tensorflow.org/tutorials/structured_data/time_series#advanced_autoregressive_model

4.1.1 Characteristics

Pros:

- Large quantity of measured streamflow and meteo data
  -> no need of meteo forecasts (that are not available from a long time)
- Architecture relatively easy to understand and the construction of sequences is straightforward
- Would use all measures until time t to produce the predictions +1 to +24h, not only the +24h time step (as implemented by the MLP).

Cons:

- No new inputs during forecast phase?
- If the model introduces a bias on its output, this bias is multiplied by the number of point we want to predict (could be small for an +24h prediction, but not anymore on the +96h prediction)

POV:

- The idea of feeding the model its own output and producing all points of an horizon of prediction is a good point.

A correction of the bias could be possible with a model trained to correct the model outputs to bring it near the measures again?

4.2 Results

In short, one model was trained at first by me when I was not entirely understanding the package and the implementation of this model. Note that I cannot say I understood entirely the way the model was implemented in NH and how it should be used by some random user of the NH package. Anyways, without having an idea of what would we like to implement as Hydrique, I am not able to decide whether the already implemented model could be useful to us.

(The first model was achieving a good score (MAPE~7) but with some cheating unintentionally induced. This score is then not relevant.

The first results using this model are then not usable. This model is not to be discarded, but it remains mandatory to decide first how we want to use its “auto-regressive” property (I mean we could use the property only on a feature containing the streamflow, but once again, it would require major adjustments in the code – costly in time.

4.3 Concerns - Questions

Following this description and how the NH repo is constructed (mainly the sequence construction), I need to clarify some points.

- Choice of features to train, repartition Measures-Forecasts:
    - Only use measure on a sequence of L points and then let the model predict let us say 24 points using only its previous output?
    - Should the model have access to meteorological forecast during the AR-part?
- Having only its previous prediction, it seems unrealistic to have a good score after 10 predicted points, without some feedback to fix its derive or bias.
    - Once again, in the idea, feeding the model its last output to let it learn when he is good or not is a great idea, but we would have to build entirely this model, and maybe create our own sequencing too. It must be implemented in a logical way and in a way its usable in the “operational” workflow.

5 Hindcast-Forecast LSTM

Context

Due to the limitations and concerns I had trying to build the AR model, I went through the other models from the NH repo. Two of them sounded very promising, solving the problem of our sequence being composed of the same set of features for all time stamps. The two models are the “Sequential LSTM” and the “Handoff LSTM” models, both having the same base structure.

Interesting feature

Indeed, both models have as common principle, the splitting of the input sequence in two period: the hindcast and the forecast period, each having a set of parametrizable inputs (features). This simple difference makes the prediction process easier in the construction of the input sequence.

Possible method to obtain a prediction

The use I thought of was to give the model a context (or hindcast) period, where the LSTM cell would go through measures of dynamical forcings while having access to the streamflow measure, and then in the forecast period only giving the model the forecasted dynamical features, of course without the streamflow measure (no cheating). Using the architecture and specific properties of a typical LSTM model, we could ally both the memory construction in the hindcast phase and then the adaptation and update of the cell state during the forecast phase.

5.1 Description of the model

As mentioned before, using this method, we make use of having two different sets of features during the two periods onto which the cell will go.

After the LSTM cell went on the whole input sequence, the last N elements of the output list are extracted and pass by a dropout Layer (according to our config file) before going in the final Linear Layer (simple FC network) to gives a sequence of N predictions. These predictions are then evaluated and the loss is computed.

This process is common to each sequence of inputs, that are fed following a batch logic, whose parameters are given in the config file (batch organized randomly).

Fig. 2 display a diagram of the main elements of this Sequential LSTM model.

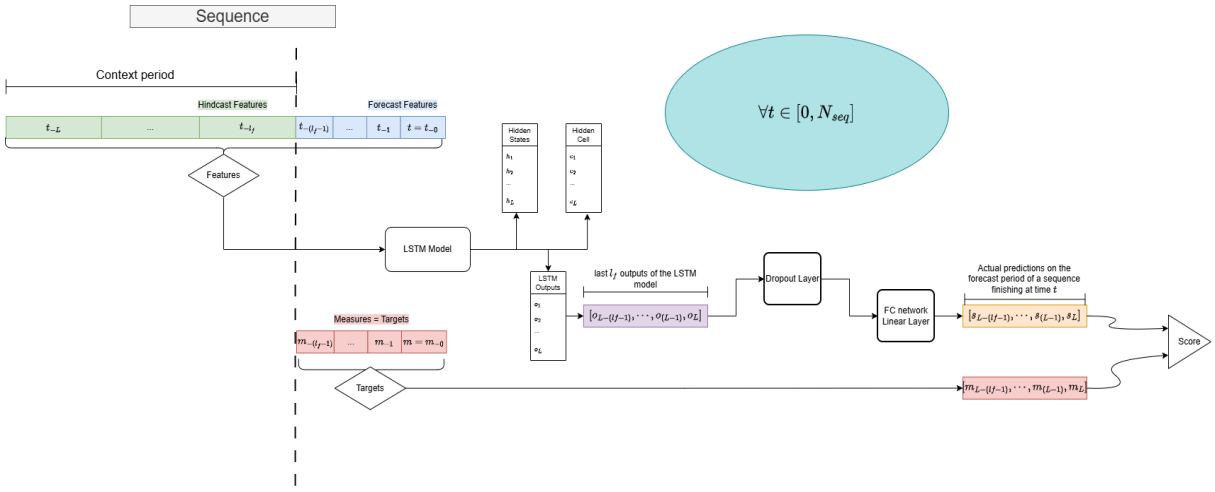


Fig. 2: Diagram of the implemented Hindcast-Forecast LSTM model

5.2 Results

Several iterations with different inputs and architectures were made to try to understand what could improve or hinder the performance of this model. I did keep a track of the major tests or improvements, but will not list them all here, while not knowing if they will be all useful for the next steps.

Among the last tests and iterations, the best MAPE score was of about 11 for the +24h point in evaluation mode and around 13 for the test period. Remark that this score is for the last element and in average over the entire horizon, is globally lower (around 7). The main parameters of this promising run are:

Length of hindcast period [hour]: 150
size of hidden parameters, number of layers of the LSTM: 128, 2
stations: stations Bruchji, Fieschertal, Visp, Eggishorn and Jungfraujoch

An attempt to improve this iteration was made by adding a lot of “rolling” features to ours. What I mean by rolling features is sums or averages over a window of 6, 24 hours of our used features. This attempt was a bit disappointing, with a score over 15 for the +24h point.

This illustrates the complexity of our roadmap to improve our forecast, and once more prove that adding more features is not the absolute solution.

As personal conclusion, I think that this results of around 11-13 is not satisfying enough, but given the number of questions or decisions that needs to be answered, it remains relatively promising. I am convinced that an improvement will be possible after having solved the main questions and having determined a clear path to explore.

As second point, this score has been obtained only with meteorological forcings, it should be possible to improve it greatly by mixing in some of our RS outputs.

Once again, the operational loop must be taken into consideration and the choice of the variables to mix in is important: h_glace, streamflow simulated, etc.

5.3 Questions

Architecture of the model:

1. Only one LSTM cell run through the 2 sequences. Need to control if the order of the inputs is important, and think if we want two cells, one per type of period (if yes then try “handoff LSTM”). The need of the same number of features for both period is a constraint, not a choice -> not good.
    a. Model “handoff lstm forecast”: two diff lstm states (1 per sequence type), maybe more modularizable
2. Placement of the dropout layer (if we add one via the config) -> between lstm and head layers (current implementation) or do we prefer in the lstm layer, between the layers for example.
    a. From what I read and saw in examples online, the dropout is indeed placed between the output of the LSTM layer and the linear Head, so all good.

Implementation:

3. Difference between validation and training loss – NH bad choice (personal view). The training loss is the metric evaluated over the complete predicted horizon at each sequence. Concerning the validation loss, it is computed over the concatenated last element of each prediction over the given period. In practice, it renders impossible for me to analyse if the model is under/over-fitting. Indeed, during the training loop, the training loss is continuously decreasing (as expected, it means that our predicted sequence becomes increasingly accurate), but our validation loss is stagnating very rapidly and then oscillates (small variations). The only thing that we can extract from this is that our horizon becomes better, but the last element is not being optimized specifically.

a. I am currently trying to implement the logical fact that the validation loss is computed as the training loss. Not yet finished. It should help me to refine the structure of the model and understand if there is any under/over-fitting.

b. It gave me an interesting idea of feature to discuss: to give a parametrizable important to each element of our horizon, to give for example more weight to the last 5 elements, to see if it could help. Seemingly, it should have a small impact on the overall score, for me the whole sequence (especially the first half of the prediction is what matters the most for our +24h prediction).

Others:

4. For now, that is already enough.

6 Sequential LSTM using only RS outputs

6.1 Description

This section is dedicated to the previous model, but this time with only the RS outputs as inputs. The data used to build the DS have been taken from the disk **Y**, in the 'massa_ml' folder. This test was first to see if the data was easily usable and to explore the availability of them. The available data goes trough 1983 to 2024, that is greatly larger than the window usable with the ICON-CH1 forecasts.

These values need to be discussed and thus the results presented below are to be taken as preliminary and do not constitute a clear achievement. However, it seems that using our LSTM base model to improve the current output of our RS simulation, as a last "layer" of optimization, could lead to a very satisfactory performance, for Hydrique and our clients.

6.2 Preliminary results

As discussed above, the RS outputs are taken here as features, for both periods. In addition, the streamflow measured value is added in the hindcast period, and is replaced by null values as forecast feature.

Fig. 3 presents the score of the first try using the simple Hindcast-Forecast LSTM model with only RS outputs as features. The period of available data spanned from 1983 to 2024.

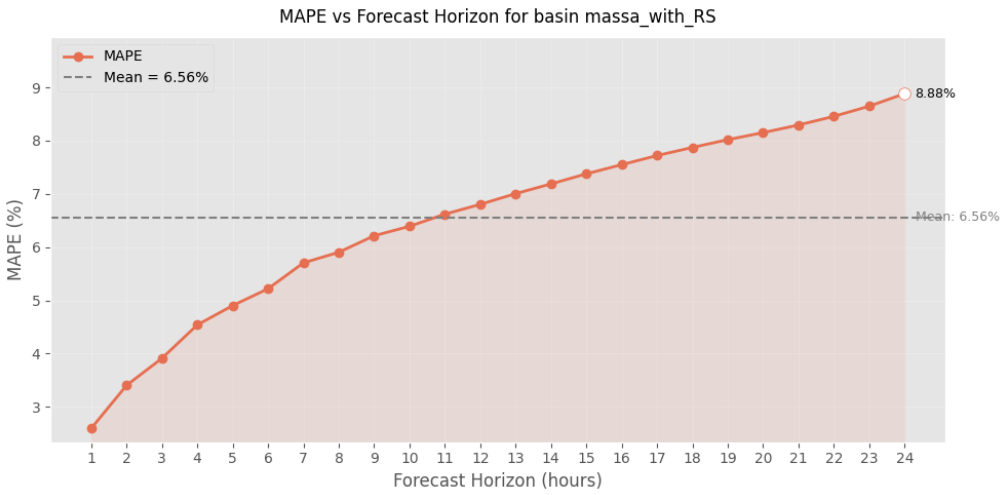


Fig. 3: Evolution of the MAPE score throughout the horizon of prediction for the sequential LSTM using RS outputs only.

For now, no try to implement both meteorological dynamical forcings and RS output as features for the model has been tested. Pending on discussion about “next steps” and “stations to use”.

7 NH – Opinion and what have been done

Python package (repo) with a “structured and practical” workspace including many “already coded” models.

NOTE: Precautions are to be taken with this code, i will describe here some mistakes or inconsistencies i found until now:

- Template for the config file is not usable, several names of variables not read by the config, clearly not up to date. I created a template personally with the main parameters (personal choice).
- The code is sometimes wrong or at least not usable in the current state. For example, the hindcast/forecast sequential LSTM model described previously was modified, to better consider the two types of features and implement correctly the “counter” feature.
- The structure of the package is a bit too rigid, rendering the code not enough modularizable. To change a model architecture or one of its parameters, we often must go into the code and change the parameter ourselves. Some practical parameters are not implemented by the Config class (i think at the number of layers of our lstm right now).
  There are also other points such as the Scaler and the Loss where the code uses a self-implemented structure and classes instead of using the already existing, and working just fine, packages. (scikit-learn is perfect for Scaler and metrics already...)

Notes on NH

According to my experience, I would say that, depending on the idea we would like to implement, use NH repo is not really what could be the most optimized. What I mean is that if we only use 10% of the code and modify even only 10% of it, updating the package with our following implementations and fetching updates from their side could lead to some errors.

If their package is not usable only by creating a dataset loader class from our side, I personally think that for the future prosperity of our workflow, in the case we build a model that satisfy our need, we would better create a small python environment tuned for our use.

I have in mind that the model I tried were clearly not very advanced in the way they were implemented and the parameters must be changed in the code manually. Also concerning the dataset, we have anyways to create the class that will load the data, centre or transform it according to our expectations. The important part of the package could be the structure that it offers, but once again, we do not use most of the files. Even the dynamic learning rate had to be modified to be functional. I am ok with working with it but the simple fact that the sequences that we feed our model are “backward” is something we must consider as a “constraint” in our implementations.

7.1 Tools implemented – some of them at least

Data – selection, gathering:

RS export, csv with excel, careful that the name of the feature is in the first row, then align the data with the column date, that serves as the “index” of the csv

Data – analysis and inspection:

In a Jupyter notebook, basis analysis of the features: plot for visual comparison between measures and forecasts, basic statistical properties for math and rigorous comparison.
NAME: DB_Analysis.ipynb

Model and run parameters:

The configure a run and precise which parameter we want to use, put all the values in the config and save it in an yml formatted file (YAML). Template available in my repo.

To properly train and test our model, a template of a Jupyter notebook is also available. What this file need to run is the config file path.

Once the training and evaluation phase is finished, a function can provide the training and evaluation loss throughout the epochs and a second function allows to see the other important metrics and their evolution during training. (plot_training_curves, plot_validation_metrics)

Note: the logging and display during training has been changed (visual changes, to better capture the evolution of the losses and have a cleaner output).

Note2: For the test, it is necessary to provide the name of the model we want to test (model state saved every X epoch, parameterised with the config). For now, there is no option “SaveBest”, that would save the best model, according to a metric – validation loss by default, could be also the sum of important metrics (mse+nse+mape) – I want to implement it.

NOTE3: SaveBest v0implemented, can take val loss or sum of MAPE, NSE and MSE as metric to determine which epoch is saved as “best”. In the test phase, “from_best” = true, to test our “best” model.

Once the model has finished the testing phase – form the test output file:

Functions:

- mape_array
  compute the MAPE metric for each horizon of prediction of our output (by default only the last element of our horizon is taken)

- plot_mape_horizon
  plot the MAPE metric in function of our horizon (increasing tendency)

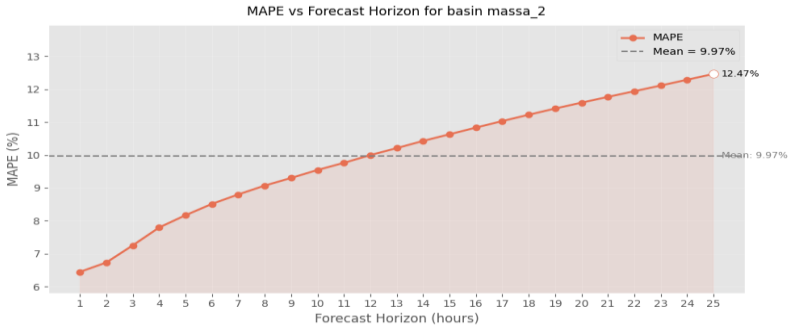


Fig. 4: Example of the evolution of a MAPE metric value for a complete horizon of prediction

- plot_forecast_24h
  interactive plot on the complete testing period, can choose which horizon we want to plot (int or list[int])

- plot_forecast_horizon_24h
  enter an issue date and it provides the forecast finishing at this date (entire horizon)

7.2 ML Typical workflow

Typical main steps:

1. Model motivation and selection
2. Data preparation
3. hyperparameters Tuning
4. Training phase and adaptation of the parameters
5. Testing phase when model seems good/promising
6. Saving the run folder

Challenge:

Each step is important and the order is important when consider what influence could have a modification onto other steps (possibly previously optimized).

References

I went through multiple documents, videos and GitHub repos online, the best are bookmarked in my browser, for now I wrote nothing here.

Ref1

Ref2

Ref3

Cette étude a été réalisée par Flavien Kohler, chef stagiaire au bureau Hydrique Ingénieurs, et supervisée par Dr Frédéric Jordan, Directeur.

Fait au Mont-sur-Lausanne, en octobre-novembre 2025.

Hydrique Engineer HJ Sàrl

Dr Frédéric Jordan

Dr Philippe Heller